Technical Interview Prep

Node.js Developer role — expected technical questions & model answers · legend: CORE required skill · PLUS nice-to-have

Focused on the **technologies named in the posting**: Node.js/TypeScript, REST & GraphQL, SQL & NoSQL, messaging patterns, NATS/RabbitMQ, Kafka, event-driven architecture, plus .NET/C# and Azure serverless. Each answer is written to be **said out loud** in ~30–60 seconds.

1 Node.js Runtime & Async CORE

Explain the event loop and its phases.

Node runs JS single-threaded on libuv's event loop. Each tick moves through ordered phases: **timers** (`setTimeout` / `setInterval`), **pending callbacks**, **poll** (I/O, where most callbacks fire), **check** (`setImmediate`), then **close** callbacks.

Between every phase — and between each macrotask — the microtask queues drain: `process.nextTick` first, then resolved Promises. So a `nextTick` can starve the loop if it re-queues itself.

`setImmediate` vs `setTimeout(fn, 0)` vs `process.nextTick`?

- `process.nextTick` — runs before the loop continues, highest priority, same tick.
- `setImmediate` — runs in the **check** phase, after poll/I/O completes.
- `setTimeout(fn,0)` — runs in **timers**; minimum ~1ms, order vs immediate is non-deterministic from the main module but immediate always wins inside an I/O callback.

Node is single-threaded — how does it handle heavy work?

I/O is offloaded to libuv's thread pool (default 4, `UV_THREADPOOL_SIZE`) so it doesn't block. But **CPU-bound** work does block the loop. Options: `worker_threads` for CPU work in-process, `child_process` / `cluster` to fan out across cores, or push the job to a queue and process it out-of-band.

How do you avoid unhandled promise rejections and callback-vs-promise mixing?

Prefer `async/await` with `try/catch`; wrap fire-and-forget promises or attach `.catch`. Listen on `process.on('unhandledRejection')` as a safety net that logs and exits, and `util.promisify` legacy callback APIs rather than nesting them.

2 TypeScript CORE

`interface` vs `type` — when do you reach for each?

`interface` for object/contract shapes I might extend or that benefit from declaration merging (e.g. augmenting Express `Request`). `type` for unions, intersections, tuples, mapped and conditional types — anything an interface can't express. In practice they overlap heavily for plain objects.

What are generics good for, in a real API?

They let one signature preserve the caller's type. A repository `findById<T>(id): Promise<T | null>`, a typed `ApiResponse<T>` envelope, or a handler wrapper that keeps request/response body types flowing end-to-end. The payoff is compile-time safety without duplicating code per entity.

`unknown` vs `any`?

`any` disables checking — it silently spreads. `unknown` is the safe top type: you can hold anything but must narrow (typeof, a guard, schema parse) before use. I use `unknown` at boundaries — parsing JSON, catch clauses — then validate into a concrete type.

How do you validate external input in TS, given types vanish at runtime?

Types are erased on compile, so a JSON body isn't actually the type it's annotated as. I validate at the edge with a schema library (Zod, io-ts, class-validator) and **infer the TS type from the schema**, so the runtime check and the static type can't drift apart.

3 REST APIs CORE

What makes an API RESTful, and how do you pick status codes?

Resource-oriented URLs (nouns), HTTP verbs for actions, statelessness, correct status semantics. Codes: `200/201/204` success, `400` bad input, `401` unauthenticated vs `403` unauthorized, `404` missing, `409` conflict, `422` validation, `429` rate-limited, `500` server, `503` dependency down.

Which verbs are idempotent, and why does it matter?

`GET`, `PUT`, `DELETE`, `HEAD` are idempotent; `POST` is not. It matters because clients, proxies and message consumers retry — an idempotent endpoint tolerates duplicate delivery. For non-idempotent creates I add an idempotency key so a retried request doesn't double-insert.

How do you version and paginate an API?

Versioning: URI (`/v1/`) for simplicity and cache-friendliness, or a header/media-type when I want cleaner URLs. Pagination: **offset/limit** for small datasets and jump-to-page; **cursor/keyset** for large or live data because offsets get slow and skip rows when items shift.

How do you secure a REST endpoint?

Auth via JWT or opaque tokens (short-lived access + refresh), enforce authz per resource, validate every input, rate-limit, set CORS explicitly, and never trust client-supplied IDs for ownership. Secrets stay in a vault/env, not code. HTTPS end to end.

4 GraphQL CORE

When is GraphQL a better fit than REST?

When clients need varying shapes of data and you want to avoid over/under-fetching, or when one screen aggregates many resources — GraphQL collapses that into a single typed request. REST stays simpler for straightforward CRUD, file transfer, and HTTP-level caching.

Explain the N+1 problem and how you fix it.

A list resolver runs, then a nested field resolver fires once per item — 1 query becomes N+1. Fix with **DataLoader**: it batches the per-item keys collected in one tick into a single backend call and caches per request. Also cap query depth/complexity so a client can't request an expensive nested explosion.

Queries vs mutations vs subscriptions?

Query reads (fields resolve in parallel). **Mutation** writes (top-level fields run serially, so ordering is defined). **Subscription** pushes updates over a persistent transport — usually WebSockets, and frequently backed by a pub/sub broker.

5 Relational Databases CORE

Walk through the isolation levels.

- **Read Uncommitted** — dirty reads possible.
- **Read Committed** — no dirty reads (Postgres default).
- **Repeatable Read** — same rows reread identically; blocks non-repeatable reads.
- **Serializable** — full isolation, as if transactions ran one at a time.

Higher isolation = fewer anomalies but more contention. I pick the lowest level that's still correct for the operation.

How do you diagnose and fix a slow query?

Start with `EXPLAIN ANALYZE` to see the real plan — look for sequential scans on big tables, bad row estimates, and expensive sorts/joins. Fixes: add or fix indexes (composite in the right column order, covering indexes), rewrite the query, update statistics, or denormalize a hot read path.

When does an index *not* help — or hurt?

It won't help low-selectivity columns (a boolean) or queries that wrap the column in a function unless it's a matching functional index. It hurts write throughput and storage, and too many indexes slow inserts/updates. Leftmost-prefix rules mean a composite index only serves queries that use its leading columns.

PostgreSQL vs SQL Server — anything you keep in mind?

Both are solid relational engines; the fundamentals transfer. Differences are mostly dialect and features: identifier quoting, `LIMIT` vs `TOP` / `OFFSET FETCH`, procedural language (PL/pgSQL vs T-SQL), and Postgres extras like rich JSONB and array types. I write portable SQL and lean on the ORM/driver for the rest.

6 NoSQL: MongoDB & Redis CORE

When do you choose MongoDB over a relational DB?

When the data is document-shaped, the schema evolves, and access patterns are known upfront so I can design documents around reads. It shines for flexible/nested data and horizontal sharding. I stay relational when I need multi-entity joins, strong cross-table constraints, and heavy transactional integrity.

Embed vs reference in Mongo?

Embed for data read together and owned by the parent (order + line items) — one read, atomic update. **Reference** for independently-queried or unbounded/large sub-data, or many-to-many, to avoid huge documents and the 16MB limit. It's a read-pattern decision, not a normalization dogma.

What do you actually use Redis for?

Caching (with TTLs), session storage, rate limiting via counters, distributed locks, leaderboards with sorted sets, and lightweight pub/sub. It's an in-memory data-structure store, so it's the go-to for anything hot-path and ephemeral.

Common caching pitfalls?

Stampede — many misses hit the DB at once when a key expires (mitigate with locks/jitter). **Stale data** — pick a strategy: TTL, or invalidate on write. And a cache is not durable, so never treat it as the source of truth.

7 Messaging Patterns CORE

Explain pub/sub, request/reply, queue routing, and topics.

- **Pub/Sub** — publisher emits, all subscribers to that subject receive a copy; publisher doesn't know consumers.
- **Request/Reply** — sender includes a reply address and awaits a response; RPC-style over a broker.
- **Queue routing / work queues** — competing consumers on one queue, each message goes to exactly one worker → load balancing.
- **Topics** — routing by pattern/subject hierarchy so subscribers filter to what they care about.

at-most-once vs at-least-once vs exactly-once?

At-most-once: no redelivery, may lose messages. **At-least-once**: redelivers until acked, so duplicates happen — the common default. **Exactly-once** end-to-end is hard/expensive; in practice we do at-least-once + **idempotent consumers** to get the same effect.

How do you make a consumer idempotent?

Give each message a stable ID and track processed IDs (a dedupe table or Redis set), or design the write to be naturally idempotent (upsert by key, conditional update). So a redelivery is a no-op instead of a double-apply.

What's a dead-letter queue?

A side queue for messages that repeatedly fail or exceed a retry/TTL limit. It keeps a poison message from blocking the queue and preserves it for inspection, alerting, and manual replay rather than silently dropping it.

8 Message Brokers: NATS & RabbitMQ CORE

How does RabbitMQ route messages?

Producers publish to an **exchange**, not a queue. The exchange type decides routing: `direct` (exact routing key), `topic` (wildcard patterns), `fanout` (broadcast to all bound queues), `headers`. Bindings connect exchanges to queues; consumers read from queues and ack.

NATS core vs JetStream?

Core NATS is fire-and-forget pub/sub — extremely fast, at-most-once, no storage. **JetStream** adds persistence, at-least-once delivery, replay, and consumer state on top. So core for ephemeral signalling, JetStream when you need durability or guaranteed delivery.

RabbitMQ or NATS — how would you choose?

NATS: very low latency, simple ops, great for high-throughput internal messaging and request/reply. RabbitMQ: mature, rich routing, per-message TTL, priorities, well-understood DLQ handling. I'd match to needs — light+fast leans NATS; complex routing/enterprise features lean RabbitMQ. The posting uses both, which is common.

How do you avoid losing messages in RabbitMQ?

Durable queues + persistent messages, **publisher confirms** so the broker acknowledges receipt, manual consumer acks only after successful processing (with `prefetch` to bound in-flight work), and DLQs for failures. Persistence trades some throughput for safety.

9 Event Streaming: Kafka CORE

How is Kafka different from a traditional message queue?

Kafka is a durable, replayable **log**, not a queue that deletes on consume. Messages persist by retention policy, and consumers track their own **offset**, so multiple consumer groups read the same stream independently and you can rewind. That makes it ideal for event sourcing and multi-consumer fan-out.

Explain partitions, consumer groups, and ordering.

A topic splits into **partitions** — the unit of parallelism. Within a consumer group each partition is owned by one consumer, so adding consumers scales up to the partition count. Ordering is guaranteed **only within a partition**, so I key messages that must stay ordered (e.g. by entity ID) onto the same partition.

How do offsets and commit strategy affect delivery guarantees?

Commit **after** processing → at-least-once (crash before commit ⇒ reprocess). Commit before → at-most-once (may skip). Auto-commit is convenient but can drop or double depending on timing, so for correctness I commit manually after the work succeeds and keep the consumer idempotent.

What is consumer lag and why watch it?

Lag = latest offset minus committed offset — how far behind a consumer is. Growing lag means consumers can't keep up with producers; it's the key health metric. I address it by adding partitions/consumers, speeding processing, or batching.

10 Event-Driven Architecture CORE

What are the trade-offs of going event-driven?

Wins: loose coupling, independent scaling, resilience (a down consumer doesn't block producers), natural audit trail. **Costs**: eventual consistency, harder debugging across async hops, message ordering/duplication concerns, and more moving infrastructure. Good for decoupled domains; overkill for a simple CRUD app.

Why do message schemas and contracts matter?

Producers and consumers are decoupled in time and deployment, so the message **is** the contract. A schema (JSON Schema, Avro, Protobuf) plus a registry enforces structure and lets you evolve safely — additive/backward-compatible changes so old consumers don't break. Versioning the event is part of the design, not an afterthought.

Event sourcing vs event-driven — are they the same?

No. **Event-driven** = components communicate via events. **Event sourcing** = state is stored as an append-only sequence of events and rebuilt by replay, rather than storing current state directly. You can be event-driven without event sourcing; event sourcing often pairs with CQRS to serve reads.

How do you keep a DB write and an event publish consistent?

You can't atomically write to a DB and a broker in one transaction — that's the dual-write problem. The **outbox pattern** fixes it: write the event to an outbox table in the same DB transaction, then a relay (poller or CDC) publishes it to the broker. So the event ships if and only if the data committed.

11 Debugging & Performance CORE

How do you find a memory leak in Node?

Watch RSS/heap trend over time; if it climbs and never settles under steady load, likely a leak. Take **heap snapshots** (Chrome DevTools via `--inspect`) at intervals and diff to see what's retained. Usual culprits: unbounded caches/arrays, listeners never removed, closures holding references, timers not cleared.

A production endpoint suddenly got slow — how do you approach it?

Look at metrics/traces first to localize: is it the DB, an upstream call, the event loop, or GC? Check **event-loop lag** and CPU. Common causes: a missing index after data grew, a blocking sync call, connection-pool exhaustion, or an N+1. I isolate the layer with data before changing code.

How do you profile CPU-bound Node code?

`node --prof` or the DevTools CPU profiler / flame graph to find the hot functions. If a genuine CPU task is blocking the loop, move it to a `worker_thread` or offload it. The goal is to keep the single main thread free for I/O.

12 .NET / C# PLUS

C# `async/await` — how does it compare to Node's model?

Conceptually similar — non-blocking awaits on `Task`. Difference: .NET is genuinely multi-threaded with a thread pool, so parallelism is first-class, whereas Node is single-threaded async. In C# I also watch for sync-over-async deadlocks (`.Result` / `.Wait()`) and use `ConfigureAwait(false)` in library code.

What's dependency injection and how does .NET use it?

DI supplies a class's dependencies from outside rather than newing them internally — better testability and decoupling. ASP.NET Core has a built-in container; you register services with a lifetime — **transient** (per resolve), **scoped** (per request), **singleton** (one instance) — and they're injected via the constructor.

Coming from Node, how would you approach maintaining .NET microservices?

The architecture concepts carry over — REST, DI, config, async, containers, the same messaging brokers. I'd get productive with the tooling (the SDK, EF Core, the project/solution layout) and lean on strong typing and the framework's conventions. The posting frames .NET as maintain-and-extend with mentorship, which fits a transfer of existing backend skills.

13 Azure Functions & Serverless PLUS

What is a cold start and how do you reduce it?

When a function scales from zero, the platform spins up a new instance and initializes the runtime — that first-call latency is the cold start. Reduce it with a warm/premium plan or pre-warmed instances, smaller deployment/dependencies, and lighter runtimes. Keeping instances warm trades cost for latency.

Triggers and bindings in Azure Functions?

A **trigger** is what invokes the function — HTTP, timer, queue/Service Bus message, blob, Event Hub. **Bindings** declaratively wire inputs/outputs (read from a queue, write to a table) so you skip boilerplate SDK code. This maps cleanly onto the event-driven work in the posting — a message arrives, a function processes it.

When is serverless the wrong choice?

For steady high-volume workloads (always-on can be cheaper than per-invocation), long-running or stateful processing, very latency-sensitive paths sensitive to cold starts, or when you need fine-grained control over the runtime. It's great for spiky, event-triggered, independently-scaling tasks.

14 Microservices PLUS

How do services communicate, and when sync vs async?

Sync (REST/gRPC) when the caller needs an immediate answer — but it couples availability and can cascade failures. **Async** (broker/events) for decoupling, resilience, and load-levelling when an eventual result is acceptable. Most systems mix both; the posting explicitly wants both synchronous (REST/GraphQL) and asynchronous (brokers/Kafka).

How do you handle a transaction spanning several services?

No distributed ACID transaction — use the **saga** pattern: a sequence of local transactions, each emitting an event that triggers the next, with **compensating actions** to undo prior steps on failure. Orchestrated (a coordinator drives it) or choreographed (services react to each other's events).

How do you stop one failing service from taking down the system?

Circuit breaker to stop hammering a failing dependency, timeouts and bounded retries with backoff + jitter, bulkheads to isolate resource pools, and graceful degradation/fallbacks. Combined with async messaging, a down consumer just means messages queue up rather than callers erroring out.

How do you trace a request across many services?

Distributed tracing — propagate a **correlation/trace ID** through every hop (HTTP headers, message metadata) so logs and spans stitch into one timeline. With something like OpenTelemetry you get end-to-end latency and can pinpoint which service in the chain is slow or erroring.